

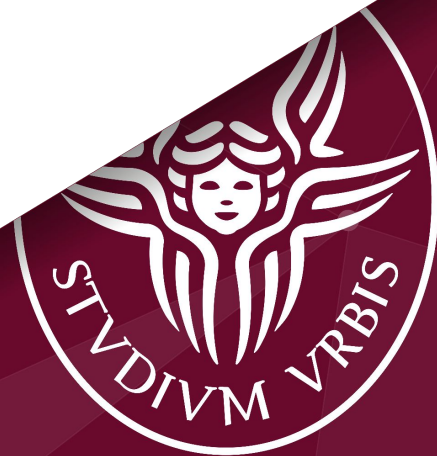


SAPIENZA
UNIVERSITÀ DI ROMA

Laboratorio di Ingegneria Informatica

Progetto Finale / Progetti Personalizzati

Prof. Roberto Navigli
Dott. Domenico Fedele



Progetto

Obiettivo complessivo

- Costruire una **pipeline end-to-end** per **acquisire e analizzare documenti** da sorgenti **web** eterogenee
- **Applicazione reale**: il vostro lavoro contribuirà direttamente alla versione chatbot dell'LLM nazionale **Minerva** (sviluppato da **Sapienza NLP** e **Babelscape**)
 - Stiamo per lanciare una versione che cerca sul Web e recupera i risultati dal motore di ricerca, li parse e li utilizza per dare una risposta aggiornata
- È una bella occasione per lavorare su un progetto concreto nel campo dell'Intelligenza Artificiale, con un impatto reale e immediato



+

 **Babelscape**



Progetto

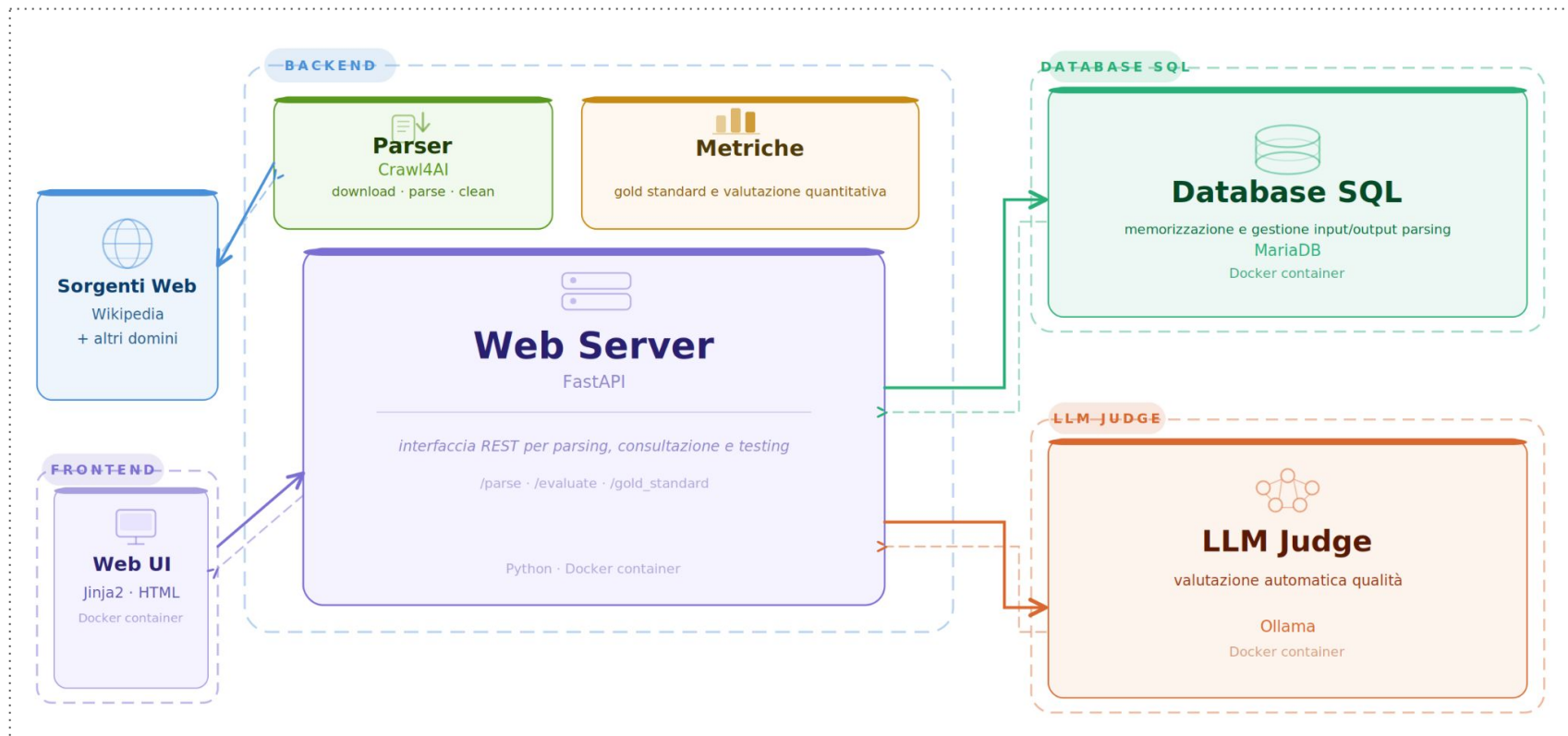
Funzionalità del sistema

- **Acquisizione di documenti** da sorgenti **web** eterogenee
- **Estrazione del testo pulito** dai documenti acquisiti tramite **parsing automatico**
- **Archiviazione** strutturata dei documenti in un **database relazionale**
- **Valutazione automatica** della **qualità** del testo
- **Giudizio qualitativo** aggiuntivo tramite un **LLM**
- Interfaccia **REST API** e **Web UI** per la consultazione, il testing e la gestione della pipeline
- **Architettura multi-container** orchestrata con Docker Compose (backend, frontend, database, LLM)



Progetto

Architettura del sistema



Progetto

Formazione dei gruppi

- Il **progetto finale** si svolge in **gruppi da 3 persone**
- I **gruppi** definiti nel [primo form pubblicato su Classroom](#) per l'esonero vanno **mantenuti** anche nel **progetto finale**
- Eventuali richieste di **gruppi** con numero **ridotto** di studenti, **ben motivate ed eccezionali**, saranno **valutate** caso per caso (non implicano una semplificazione del progetto)

Progetto

Informazioni generali

- **Consegna**
 - Il progetto completo va consegnato al massimo **8 giorni prima di ciascun appello**
 - **Esempio:** appello il 25 dicembre → consegna entro il 17 dicembre (mezzanotte circa)
 - Il progetto va consegnato su Google **Classroom** tramite l'apposito **form**, che sarà pubblicato nelle settimane precedenti alla scadenza
- Il progetto **estende** quanto fatto con l'**esonero**, che non è obbligatorio ma copre una parte sostanziale del lavoro. Chi ha sostenuto l'esonero segue le stesse regole di consegna del progetto completo, con il vantaggio di avere già una parte del lavoro svolta.
- **Accesso alla Tesi**
 - **Tutti gli studenti possono utilizzare il progetto svolto come tesi triennale**, applicando una personalizzazione che verrà illustrata successivamente
 - I migliori **progetti** dell'**esonero 1** potranno accedere a un **esonero 2/progetto di tesi personalizzato in NLP**
 - Entrambe le modalità saranno presentate nel dettaglio al termine della presentazione

Progetto



Progetto

Obiettivi

1. **Implementazione completa dei parser per tutti i domini web assegnati**
Parser dedicati per estrarre il testo rilevante dalle pagine dei domini assegnati
2. **Costruzione di un Gold Standard (GS)**
Dataset di riferimento costruito manualmente, contenente il testo corretto da confrontare con l'output del parser
3. **Valutazione automatica dei parser con metriche specifiche**
Misurazione della qualità del parsing tramite metriche quantitative
4. **Valutazione automatica dei parser con LLM-as-Judge**
Utilizzo di un LLM per validare la qualità del parsing
5. **Database**
Memorizzazione e gestione input/output parsing
6. **Esposizione funzionalità via API REST**
Endpoint per parsing, accesso ai dati e valutazione automatica
7. **Web UI Completa**
Interfaccia per gestire parsing, evaluation e DB
8. **Containerizzazione con Docker**
Ambiente multi container eseguibile in modo controllato e uniforme



Obiettivo 1: Parser per Domini Web

Task

- **Parser:** implementare un parser Python specifico per le pagine di ciascun dominio web assegnato (es. it.wikipedia.org)
- Data un'URL proveniente da uno dei **domini assegnati** il parser deve **estrarre** le seguenti **informazioni**:
 - `"url": "https://..."`,
 - `"domain": "esempio.it"`,
 - `"title": "Titolo della pagina web"`,
 - `"html_text": "Testo estratto della pagina web grezzo (HTML senza filtri), su cui è possibile rieseguire il parsing per ottenere di nuovo parsed_text"`
 - `"parsed_text": "Testo estratto pulito senza tag HTML o di altro genere, in formato MARKDOWN"`

Obiettivo 1: Parser per Domini Web

Libreria per il parsing - Crawl4AI

- Per il **download** e il **parsing** delle **pagine web** potete utilizzare la **libreria** [Crawl4AI](#) con **configurazione personalizzata** per ogni **dominio**
- **Crawl4AI** è una **libreria Python open-source** progettata per il **web scraping** orientato all'AI
- Trasforma qualsiasi **pagina web** in dati **strutturati** (**Markdown**, **JSON**, **screenshot**, **HTML**) **pronti** per essere consumati da **modelli linguistici** (LLM)
- Sfruttate la libreria per estrarre direttamente il **Markdown** ed eventualmente **ripulirlo** ulteriormente

Obiettivo 1: Parser per Domini Web

Libreria per il parsing – Crawl4AI

- Potete utilizzare **librerie aggiuntive**, se ritenute utili, come ad esempio [BeautifulSoup](#)
- Tuttavia, è **preferibile** e particolarmente apprezzato l'impiego di **Crawl4AI** per il **parsing**, individuando una **configurazione efficace** e adatta al caso d'uso del progetto.

Obiettivo 1: Parser per Domini Web

Crawl4AI - Installazione

- Installare la libreria
pip install -U crawl4ai
- Setup del browser (Chromium scaricato automaticamente)
crawl4ai-setup
- Verificare l'installazione
 - **crawl4ai-doctor**
 - **python -c "import crawl4ai; print('OK')"**
- Per eventuali problematiche potete consultare la [documentazione](#)

Obiettivo 1: Parser per Domini Web

Crawl4AI - Esempio di download e parsing

- [BrowserConfig](#)

Configura il browser
(headless, proxy, viewport)

- [CrawlerRunConfig](#)

Configura la singola richiesta. Agire sulla configurazione per migliorare l'output del parsing della libreria

- [AsyncWebCrawler](#)

Il motore del crawler; usato come context manager apre e chiude il browser in modo sicuro. `.arun()` visita l'URL e restituisce il contenuto estratto usando la config scelta. Dovete agire anche sull'output del parsing per migliorare la qualità.

```
import asyncio
from crawl4ai import AsyncWebCrawler, BrowserConfig, CrawlerRunConfig, CacheMode

async def main():

    # configura il browser (headless = senza finestra visibile)
    browser_cfg = BrowserConfig(headless=False)

    # configura la richiesta (BYPASS = scarica sempre dalla rete, ignora cache)
    crawler_cfg = CrawlerRunConfig(cache_mode=CacheMode.BYPASS)

    # apre il browser e lo chiude automaticamente alla fine del blocco
    async with AsyncWebCrawler(config=browser_cfg) as crawler:

        # visita la pagina e aspetta che il crawl sia completato
        result = await crawler.arun(
            url="https://it.wikipedia.org/wiki/Minerva",
            config=crawler_cfg
        )

        # result.success          - True se il crawl è andato a buon fine
        # result.error_message   - messaggio d'errore in caso di fallimento
        #
        # alcuni output disponibili:
        # result.markdown         - testo in Markdown, pronto per LLM e RAG
        # result.cleaned_html     - HTML ripulito da script, stili e rumore
        # result.html             - HTML completo della pagina (non ripulito)
        print(result.markdown)

# avvia il programma asincrono
asyncio.run(main())
```

Obiettivo 1: Parser per Domini Web

Crawl4AI - Esempio download HTML

```
async def fetch_raw_html(url: str) -> str:
    """Usa Crawl4AI solo per ottenere l'HTML grezzo, senza post-processing."""
    browser_config = BrowserConfig(
        browser_type="chromium",
        headless=True,
    )
    fetch_config = CrawlerRunConfig(
        wait_until="domcontentloaded",
        process_iframes=False,
        remove_overlay_elements=False,
    )
    async with AsyncWebCrawler(config=browser_config) as crawler:
        result = await crawler.arun(url=url, config=fetch_config)
    if not getattr(result, "success", True):
        raise RuntimeError(getattr(result, "error_message", "Fetch fallito"))
    html = getattr(result, "html", "") or ""
    if not html:
        raise RuntimeError("Crawl4AI non ha restituito HTML.")
    return html
```

- È possibile scaricare solo il contenuto HTML senza alcun tipo di processamento, utile in fase di **evaluation** e per la costruzione del GS

Obiettivo 1: Parser per Domini Web

Crawl4AI - Esempio di parsing da stringa HTML

```
async def reparse_html(html: str, site: Site) -> str:
    """
    Parsa l'HTML grezzo usando Crawl4AI.
    """
    # estrae la configurazione di parsing specifica per il sito
    run_config = build_crawler_run_config(site)

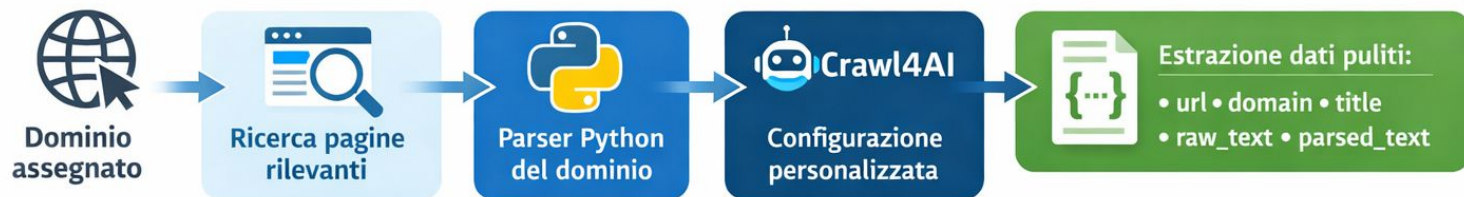
    # parse l'HTML grezzo usando Crawl4AI, con la configurazione specifica per il sito
    async with AsyncWebCrawler(config=build_browser_config()) as crawler:
        result = await crawler.arun(
            url=f"raw:{html}",
            config=run_config,
        )

    # estrai il "best content" dal risultato del parsing, usando la logica specifica per il sito
    return extract_best_content(result, site)
```

- È possibile **parsare** contenuto **HTML statico** precedentemente **scaricato**, questo vi permette anche di gestire al meglio le fasi di **evaluation** per un sito che cambia molto rapidamente
- La libreria Crawl4AI ha numerose **funzionalità** che vi consiglio di esplorare nella sua [documentazione](#)

Obiettivo 1: Parser per Domini Web

Pipeline di Parsing



1. **Selezione** manuale delle **pagine** più **rilevanti** del **dominio** assegnato
2. **Parser Python** dedicato per ciascun dominio
3. **Crawl4AI** con configurazione personalizzata per il dominio specifico (modificare **CrawlerRunConfig** e/o l'output del parsing)
4. Estrazione di **metadati** e **testo pulito** dall'output di Crawl4AI
5. **Output** strutturato: **url, domain, title, html_text, parsed_text**

Assegnazione dei domini da parsare

- Ogni gruppo dovrà **parsare pagine** provenienti da **domini web specifici** (es. it.wikipedia.org, <https://www.reddit.com>, <https://stackoverflow.com> e altri)
 - NOTA: non dovete parsare la home page del sito, ma solo le pagine informative
- Il **parsing** delle pagine provenienti dal dominio [wikipedia.org](https://it.wikipedia.org) è uguale per tutti In [inglese](#) se, ordinando i cognomi del gruppo in ordine alfabetico, il primo cognome inizia con una lettera “dispari”, ovvero A, C, E, G, I, K, M, O, Q, S, U, W, Y; in [italiano](#) altrimenti
- **3 domini aggiuntivi** saranno assegnati **casualmente** ad ogni gruppo:
 - Wikipedia e TUTTI i domini aggiuntivi devono essere **parsati** correttamente (implementare 4 parser, sfruttare la programmazione ad oggetti)
 - **Più domini** riuscirete a parsare correttamente **maggiore** sarà la **valutazione**
 - I **domini** aggiuntivi **assegnati** per l’esonero sono gli **stessi** del progetto finale (vedere [Assegnazione Domini](#))
 - Per i **nuovi gruppi** i **domini aggiuntivi** saranno **assegnati** mediante un apposito **script**

Assegnazione dei domini da parsare

Script automatico per i NUOVI GRUPPI

- Script automatico per l'assegnazione dei domini **SOLO PER I NUOVI GRUPPI**:
[Link allo script: domains_by_surname.py](#)
- Rimpiazzate:
"Cognome 1", "Cognome 2", "Cognome 3"
con i vostri cognomi ed eseguite:
"python domains_by_surname.py"
il codice stamperà la vostra Wikipedia e l'ID
dei 3 domini aggiuntivi
- Ricavate il nome del dominio dalla lista
domini assegnati usando gli ID che vi ha
restituito lo script:
[Link: Lista Domini & URL di Esempio](#)

```
def wikipedia_lingua(cognomi: list[str]) -> str:
    """Ritorna 'en' o 'it' in base all'iniziale del primo cognome ordinato."""
    primo = sorted(c.strip().upper() for c in cognomi)[0]
    return "en" if primo[0] in LETTERE_DISPARI else "it"

def assegna_domini(cognomi: list[str]) -> list[int]:
    """Ritorna 3 numeri distinti in [1, DOMINI] derivati deterministicamente dai cognomi."""
    # 1. Normalizza (maiuscolo, no spazi), ordina e unisci con "|"
    cognomi_norm = sorted(c.strip().upper() for c in cognomi)
    chiave = "|".join(cognomi_norm)
    # 2. SHA-256: 32 byte, distribuzione uniforme, ordine-invariante
    digest = hashlib.sha256(chiave.encode("utf-8")).digest()
    # 3. Scorri la digest a finestre di 2 byte; mappa ogni coppia in [1, DOMINI]
    numeri = []
    i = 0
    while len(numeri) < 3 and i < len(digest) - 1:
        valore = int.from_bytes(digest[i:i+2], "big") % DOMINI + 1
        if valore not in numeri:
            numeri.append(valore)
        i += 1
    return numeri
```

Obiettivo 2: Gold Standard per Domini Assegnati

Cos'è un Gold Standard?

- Un **Gold Standard (GS)** è un insieme di **dati** di riferimento, costruito e verificato **manualmente**, considerato come **corretto**
- Serve a **valutare** la **qualità** dei risultati **prodotti** da un sistema automatico, fornendo un termine di confronto affidabile
- Si **confronta** l'**output del sistema** con il **GS**, così da misurare in modo oggettivo quanto il risultato prodotto sia corretto

Obiettivo 2: Gold Standard per Domini Assegnati

Gold Standard per il parsing di pagine Web

- **Gold Standard (GS):** per ogni **dominio assegnato**, selezionare almeno **10 URL** rappresentative (pagine di tipo diverso, con contenuti vari, no home page)
- **Maggiore** sarà la **qualità del GS** **maggiore** sarà la **valutazione**. Un GS di buona qualità è quello che ci permette di valutare al meglio un parser
- **Per ogni URL:** aprire la pagina nel **browser**, copiare **manualmente** il solo **testo informativo** (no pubblicità, contenuti collaterali o ripetuti in ogni pagina, ecc.)
 - **Includere:** titolo e corpo dell'articolo
 - **NON includere:** menu di navigazione, footer, sidebar, banner pubblicitari
- Utilizzare il **GS** per valutare la **qualità dei vostri parser**

Obiettivo 2: Gold Standard per Domini Assegnati

Formato del Gold Standard

- Salvare i **GS** in file **JSON**, uno per ogni dominio e nel vostro **Database**

Un'entry del vostro **GS** contiene le seguenti informazioni:

- `"url": "https://..."`,
- `"domain": "esempio.it"`,
- `"title": "Titolo della pagina web"`,
- `"html_text": "Testo estratto della pagina web grezzo (HTML senza filtri), su cui è possibile rieseguire il parsing per ottenere di nuovo parsed_text da confrontare con gold_text, molto utile in fase di evaluation" (vedere download html e parsing da stringa HTML)`
- `"gold_text": "Testo estratto pulito senza tag HTML o di altro genere, non in formato MARKDOWN"`

Obiettivo 2: Gold Standard

Esempio estrazione info - Wikipedia

Indice

nascondi

Inizio

Titoli e ruoli

Cultura

Culto

Iconografia

Calculus Minervae

Note

Voci correlate

Altri progetti

Collegamenti esterni

WIKIPEDIA

25 anni di enciclopedia libera

Q Cerca in Wikipedia

Ricerca

Minerva

Voce Discussione

79 lingue

Leggi Modifica Modifica wikitesto Cronologia Strumenti

30 marzo: Wikipedia non può ancora essere venduta

Ci dispiace per i ripetuti tentativi, ma oggi è lunedì, 30 marzo e abbiamo ancora bisogno di aiuto. Se tutti coloro che leggono questo messaggio donassero oggi 2,75 € raggiungeremmo il nostro obiettivo subito. La maggior parte non dona. Wikipedia è gestita da un'organizzazione non-profit, non da un miliardario. Quindi se Wikipedia ti ha dato una conoscenza che vale almeno 2,75 €, per favore dona. Ogni contributo è utile, che siano 2,75 € o 25 €.

Dona 2,75 €

Dona un altro importo

FORSE DOPO

HO GIÀ DONATO

CHIUDI

Disambiguazione – Se stai cercando altri significati, vedi [Minerva \(disambigua\)](#).

Minerva (in latino *Minerva*) è la **divinità romana** della lealtà in lotta, delle virtù eroiche, della guerra giusta (guerra per giuste cause o per difesa), della **saggezza**, delle strategie, ed è riconosciuta anche protettrice degli **artigiani**.

Le sue origini discendono dagli **Etruschi**, che molto presto la fusero con **Athena**, suo corrispettivo nella **mitologia greca**. Minerva era venerata anche dagli antichi italici, come dimostra il santuario arcaico di **Lavinium**.

Come per Athena anche per Minerva l'animale sacro è la **civetta**, talvolta il gufo. Secondo il mito, era figlia di **Giove**, nata dalla testa di quest'ultimo.

Titoli e ruoli

[modifica] [modifica wikitesto]

Il termine *Minerva* fu probabilmente importato dagli **Etruschi** che la chiamavano **Menrva**. I Romani ne confusero il nome straniero con il loro lemma *mens* (mente), dal momento che la dea governava non solo la guerra, ma anche le attività intellettuali.

Minerva era figlia di Giove e, dopo l'assimilazione con la greca Athena, il mito della sua nascita la fece frutto dell'unione del re degli dei con la ninfa **Metis**.

Minerva, dettaglio del *Trionfo della Virtù* di Andrea Mantegna (1494-1502)

Fai una donazione Registrati

Aspetto nascondi

Modalità completano (Baby Giochi)

☒ Disabilitata

☐ Abilitata

Scopri di più sulla Modalità completano

Testo

☐ Piccolo

☒ Standard

☐ Grande

Larghezza

☒ Standard

☐ Largo

Colore (beta)

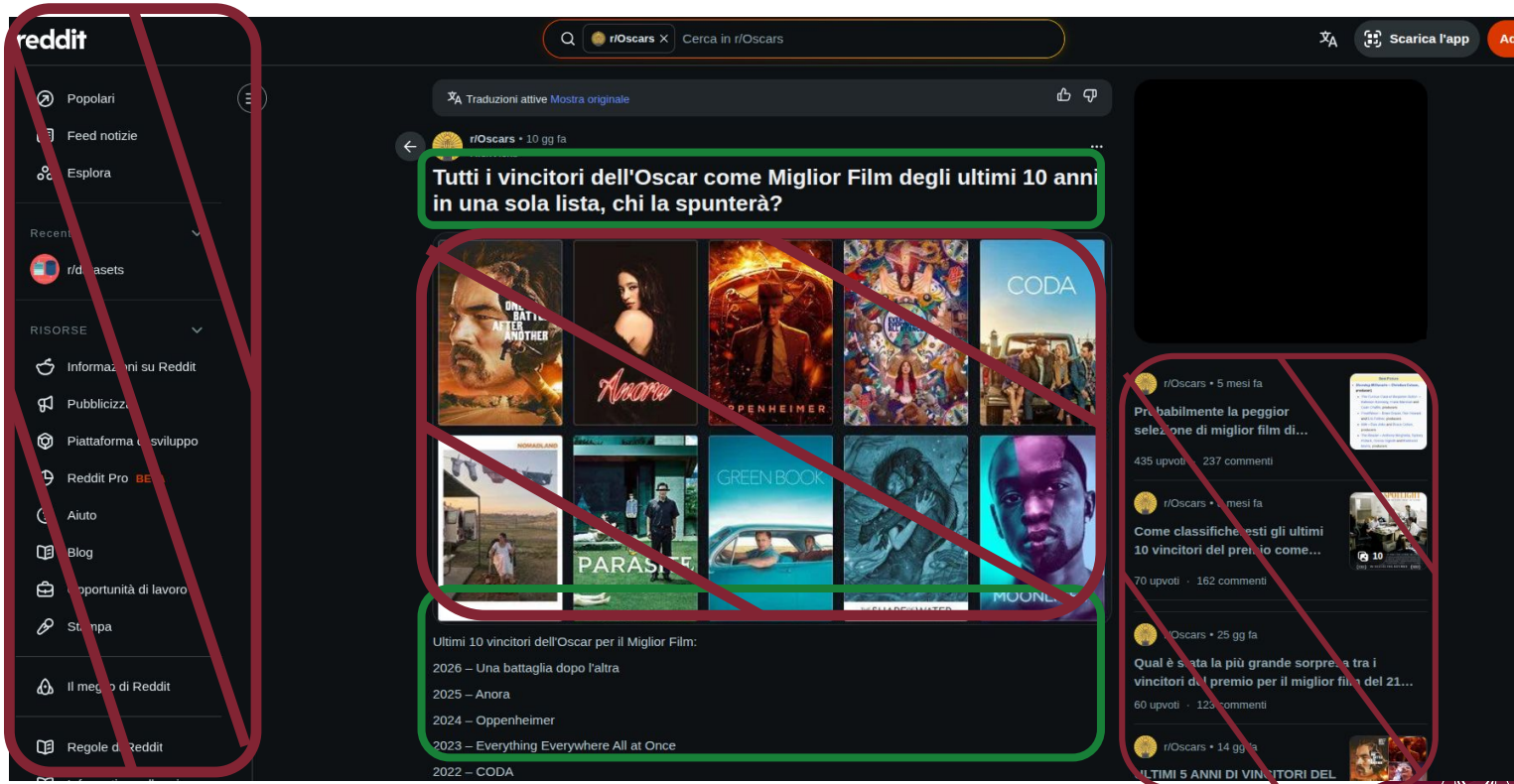
☐ Automatico

☒ Chiaro

☐ Scuro

Obiettivo 2: Gold Standard

Esempio estrazione info - Wikipedia



Obiettivo 2: Gold Standard

Esempi di GS

- Esempi di **elementi** del **GS** per alcune pagine di **Wikipedia**
- Dominio 1: en.wikipedia.org
 - Pagina 1: en.wikipedia.org/wiki/Minerva -> [GS](#)
 - Pagina 2: en.wikipedia.org/wiki/BabelNet -> [GS](#)
- Dominio 2: it.wikipedia.org
 - Pagina 1: it.wikipedia.org/wiki/Minerva -> [GS](#)
 - Pagina 2: it.wikipedia.org/wiki/BabelNet -> [GS](#)
- Per i **domini aggiuntivi** dovete **valutare voi** quali sono le **informazioni rilevanti** per una **pagina web**, ma in caso di **dubbi** potete **scrivere su Classroom**

Obiettivo 2: Gold Standard

Trovare pagine interessanti per domini aggiuntivi

- Per trovare pagine interessanti per uno specifico dominio, si possono seguire gli **esempi** presenti nell'[elenco completo dei domini](#) condiviso su Classroom
- Per trovare **altre tipologie di pagine** interessanti si può usare l'operatore **"site: <sito web>"** di Google per **limitare la ricerca sul browser ad un sito specifico**
- L'idea è cercare **contenuti utili** e informativi, adatti ad un **LLM** che cerca sul Web per rispondere ad una domanda di un utente
- **Alcuni Esempi** con Wikipedia in Italiano (it.wikipedia.org):
 - ["città italiane site:it.wikipedia.org"](#)
 - ["energia nucleare site:it.wikipedia.org"](#)
 - ["Dante Alighieri site:it.wikipedia.org"](#)

Obiettivo 3: Valutazione Parser con Metriche

Valutare Qualità Parsing

- Valutare la qualità dell'output prodotto dai parser implementati
- La valutazione sarà effettuata in prima battuta tramite metriche quantitative implementate da voi, usate come indicatori oggettivi della qualità del parsing
- I Gold Standard (GS) prodotti saranno utilizzati come dataset di test per il confronto con l'output del parser
- Una metrica di valutazione è obbligatoria e uguale per tutti: token_level_eval
- Ci aspettiamo che esplorate e implementiate in autonomia ulteriori tecniche di valutazione (sperimentate, cercate online, chiedete agli LLM!)

Obiettivo 3: Valutazione Parser con Metriche

Valutare Qualità Parsing

- **Evaluation obbligatoria** da implementare “**token_level_eval**”:
 - La valutazione avviene a **livello di token**
 - parole separate per spazio in lowercase
 - senza markdown, in questa fase la parte rilevante da valutare è il testo estratto
 - **tokens_estratti** = insieme dei token nel testo prodotto dal parser
 - **tokens_gs** = insieme dei token nel testo del Gold Standard
 - **Metriche**
 - **Precision** = $|tokens_estratti \cap tokens_gs| / |tokens_estratti|$
Tra i token trovati dal modello, quanti sono davvero corretti?
 - **Recall** = $|tokens_estratti \cap tokens_gs| / |tokens_gs|$
Tra tutti i token corretti possibili, quanti il modello è riuscito a trovare?
 - **F1** = $2 \times precision \times recall / (precision + recall)$
Media armonica tra precision e recall, utile per avere un unico indicatore bilanciato

Obiettivo 3: Valutazione Parser con Metriche

Come Interpretare la '*token_level_eval*' F1

<i>Fascia</i>	<i>Range F1</i>	<i>Interpretazione</i>
Scarso	< 0.6	Il parser perde contenuto rilevante o introduce molto rumore
Medio	0.6 – 0.80	Contenuto principale presente ma con omissioni o boilerplate non rimosso
Buono	> 0.80	Il parser cattura la maggior parte del testo rilevante con poco rumore

NOTA: data la semplicità di questa metrica, "Buono" è condizione necessaria ma non sufficiente per considerare il parser affidabile. Verificate anche manualmente.

Obiettivo 3: Valutazione Parser con Metriche

Come gestire confronto tra output in Markdown e Gold Standard

- Essendo l'output del parser in Markdown e il GS in raw text vanno **normalizzati** prima della valutazione rimuovendo markdown e link
- Pur non coprendo tutti i casi esistenti, una possibile soluzione per **rimuovere il Markdown SOLO in fase di valutazione** con la *token_level_eval* è la seguente:

```
def remove_markdown(md: str) -> str:
    """
    Rimuove il Markdown da una stringa, restituendo solo il testo pulito.
    Usa la libreria mistune per convertire il Markdown in HTML, poi BeautifulSoup per estrarre solo il testo.
    """
    html = mistune.html(md)
    soup = BeautifulSoup(html, "html.parser")
    # rimuove i tag lasciando il testo esattamente in-place (nessun separatore aggiunto, mantiene punteggiatura)
    for tag in soup.find_all(True):
        tag.unwrap()
    text = re.sub(r'[ \t]+', ' ', str(soup)) # collassa spazi orizzontali (non \n)
    text = re.sub(r'\n+', '\n', text) # collassa nuove linee multiple in una sola
    return text.strip()
```

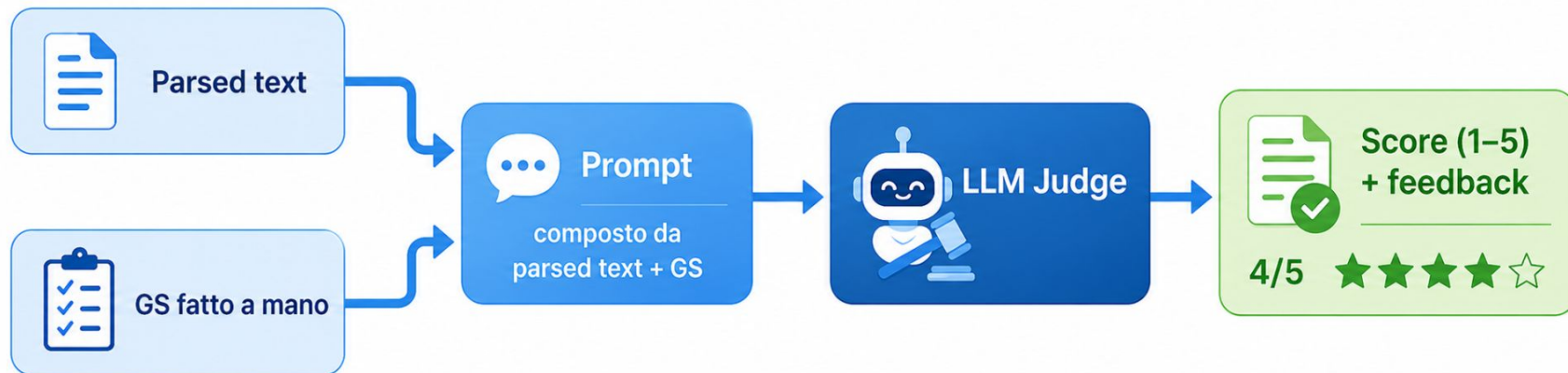
Obiettivo 4: Valutazione Parser con LLM

LLM-as-Judge

- L'idea dell'**LLM-as-Judge** è di utilizzare un **Large Language Model** come giudice automatico per **valutare** la qualità del **testo** estratto dai **parser**
- Il **funzionamento** è il seguente:
 - Il **testo parsato** e il **Gold Standard** vengono inviati all'**LLM** tramite un **prompt strutturato**
 - Il **prompt** chiede al modello di **valutare** la qualità del **testo estratto confrontandolo** con il **Gold Standard**, **identificando** eventuali **problemi** come boilerplate residuo, testo troncato o rumore
 - L'**LLM restituisce** un **giudizio** strutturato (es. score e feedback)

Obiettivo 4: Valutazione Parser con LLM

Pipeline di Valutazione con LLM



Obiettivo 4: Valutazione Parser con LLM

Esempio di Prompt per il Judge

- Il **prompt** deve essere **strutturato** in modo da **guidare l'LLM** a restituire un giudizio in formato **JSON**.
- Un **esempio** molto semplice e generico:

```
" Valuta la qualità del seguente testo estratto da una pagina web.  
Testo estratto dal parser:  
{parsed_text}  
Testo di riferimento (Gold Standard):  
{gold_text}  
Rispondi SOLO con un JSON nel seguente formato:  
{  
  "score": <tra 1 e 5>,  
  "feedback": <breve descrizione della qualità del testo>  
}"
```

- È obbligatorio **gestire il caso** in cui l'LLM non rispetti il formato atteso, prevedendo un **fallback** appropriato

Obiettivo 4: Valutazione Parser con LLM

Tecnologie

- Per far girare il modello in locale è **obbligatorio** utilizzare **Ollama**, che verrà approfondito a lezione
- I modelli da utilizzare devono essere **modelli piccoli compatibili** con la CPU del vostro PC
- **Non aspettatevi risultati perfetti**: con un modello più grande la qualità migliorerà sensibilmente, ma per il progetto base sono più che sufficienti



Obiettivo 4: Valutazione Parser con LLM

Modelli Da Utilizzare

- Abbiamo selezionato **5 modelli** adatti ad esecuzioni su CPU (anche di un notebook):

Modello	Developer	Parametri	RAM stimata	Comando Ollama
<u>gemma4:e2b</u>	Google	2.3B attivi	~2GB	ollama pull gemma4:e2b
<u>llama3.2:3b</u>	Meta	3B	~2GB	ollama pull llama3.2:3b
<u>phi4-mini</u>	Microsoft	3.8B	~3GB	ollama pull phi4-mini
<u>qwen3:4b</u>	Alibaba	4B	~3GB	ollama pull qwen3:4b
<u>ministral-3:3b</u>	Mistral AI	3.85B	~3GB	ollama pull ministral-3:3b

- Sta a voi **valutare** quale **modello utilizzare** tra questi **5** in base alla **qualità del giudizio** ottenuto
- Vi consigliamo di fare **test** su tutti e **5 i modelli** prima di scegliere quello definitivo da usare nel progetto.

Obiettivo 5: Database

Scopo del Database

- Il **database relazionale (SQL)** ha il ruolo di **centralizzare** e rendere **persistenti** tutte le **informazioni prodotte** dalla **pipeline di parsing**:
 - I **contenuti grezzi** scaricati dalle sorgenti web (**HTML**), utili in fase di **evaluation**
 - I **testi parsati** e puliti prodotti dai parser
 - I **Gold Standard** costruiti manualmente
 - I risultati delle **valutazioni** automatiche con **metriche**
 - I **giudizi** qualitativi prodotti dall'**LLM Judge**
- **Senza un database**, ogni **risultato** sarebbe **temporaneo** e andrebbe perso al riavvio del sistema oppure andrebbe gestito **tramite file**
- Il **DB** permette di **consultare**, **confrontare** e **analizzare** i dati nel tempo tramite le **API REST** e la **Web UI**

Obiettivo 5: Database

Schema del Database

- Lo **schema del database** è a vostra **discrezione** fatta eccezione per **2 tabelle** e sarà **oggetto di valutazione**
- Un possibile **esempio** di come potrebbe essere **organizzato il DB**:
 - **Risorse web**: URL, dominio, contenuto HTML grezzo
[tabella obbligatoria: **web_resources**]
 - **Documenti parsati**: output del parser
 - **Gold Standard**: testo di riferimento costruito manualmente
[tabella obbligatoria: **gold_standard**]
 - **Valutazioni**: risultati delle varie valutazioni automatiche
 - **Giudizi LLM**: score e verdetto del judge
- Le **tabelle** devono essere **collegate** tra loro tramite **primary key** e **foreign key**, garantendo l'**integrità referenziale** dei dati

Obiettivo 5: Database

Schema del Database – *web_resources* & *gold_standard*

- Per rendere i **test automatici** robusti, si definiscono **2 tabelle obbligatorie con nomi fissi**:

- **web_resources** (contiene le risorse web scaricate da utilizzare in locale)
 - *url*: URL della pagina, chiave primaria
 - *domain*: dominio della pagina
 - *title*: titolo della pagina
 - *html_text*: contenuto HTML grezzo
 - *created_at*: data di acquisizione
- **gold_standard** (contiene il testo di riferimento costruito manualmente)
 - *url*: chiave primaria e esterna verso *web_resources*
 - *gold_text*: testo corretto estratto manualmente
 - *created_at*: data di creazione

- La **relazione** tra le due tabelle è **uno-a-uno**: ogni entry del gold standard è associato a una risorsa web
- I **test automatici** **possono alterare** queste tabelle e **sfruttarle** per forzare l'utilizzo dell'**HTML locale**

- Le **altre tabelle** del progetto restano **libere** e possono essere progettate dal gruppo in base alla propria architettura

Obiettivo 5: Database

Popolamento Iniziale del Database

- **All'avvio del sistema, il database deve essere popolato con i dati necessari al funzionamento della pipeline di parsing:**
 - I **Gold Standard** costruiti manualmente devono essere **caricati automaticamente** nel DB, in modo che siano subito **disponibili** per le **valutazioni** tramite le API REST e la Web UI
 - L'**HTML grezzo** scaricato durante la **costruzione** del **Gold Standard** deve essere anch'esso salvato nel DB, in modo da poter **rieseguire il parsing** in qualsiasi momento e **confrontare** il risultato con il **gold_text**
 - Il **caricamento** deve avvenire tramite uno **script di inizializzazione** che viene eseguito automaticamente al **primo avvio**

NOTA: i file **JSON** del **Gold Standard** prodotti, contenenti sia il **gold_text** che l'**HTML grezzo**, devono essere **inclusi nel progetto**

Obiettivo 5: Database

Tecnologie

- Per la gestione del database è obbligatorio utilizzare:
 - **MariaDB** come sistema di gestione del database relazionale
 - **MariaDB Connector/Python** come libreria per l'interazione tra Python e il database
- **MariaDB** girerà in un container **Docker** dedicato, coordinato tramite Docker Compose insieme agli altri container del sistema



Obiettivo 6: API REST

Overview API REST

Implementare con **FastAPI** un **WEB Server** per accedere ai seguenti endpoint:

- **POST /parse:** esegue il parser per un documento
- **GET /domains:** restituisce la lista dei domini assegnati
- **GET /gold_standard:** restituisce il gold standard per un documento
- **GET /gold_standard_urls:** restituisce tutti gli URL del GS
- **POST /evaluate:** dato risultato parsing e gs restituisce le metriche di evaluation
- **POST /evaluate_judge:** dato risultato parsing e gs restituisce l'evaluation del judge
- **GET /full_gs_eval:** restituisce l'evaluation aggregata su tutto il GS
- **POST /add_web_resource & /add_gold_standard:** aggiungono dati al DB
- **DELETE /web_resource & /gold_standard:** rimuovono dati dal DB
- **GET /db_stats:** restituisce stats generali calcolate dai dati nel DB
- **GET /db_schema:** restituisce lo schema del DB
- **GET /status:** restituisce lo stato dei componenti del sistema

Obiettivo 6: API REST

Requisiti API REST

- Usare **FastAPI** e **Pydantic** per validazione I/O
- I **formati di input/output** devono **rispettare** esattamente la **specifica**, perché saranno **testati automaticamente**
- **Gestire** esplicitamente **errori** come dominio non supportato e URL irraggiungibile

Obiettivo 6: API REST

POST /parse

- Seleziona automaticamente il parser corretto in base al dominio in "url" ed esegue il **parsing** sull'HTML scaricato al momento. Se viene passato "**local = true**", usa la pagina nel DB senza effettuare il download (usare "url" per cercarla nel DB)
- **Input:** {"url": str, "local": Optional[bool]}
- **Output:** oggetto JSON nel formato indicato
- **Errori:**
 - Se il dominio non è supportato
 - Se l'URL è irraggiungibile/non è nel DB

// OUTPUT - JSON

```
{  
  "url": "https://...",  
  "domain": "esempio.it",  
  "title": "Titolo estratto dalla pagina web",  
  "html_text": "Html estratto dalla  
                pagina web, input del parser",  
  "parsed_text": "Testo pulito, output del  
                parser di riferimento, in  
                formato MARKDOWN"  
}
```

Obiettivo 6: API REST

GET /domains

- Restituisce la lista dei domini supportati dal sistema (es. it.wikipedia.org)
- **Input:** nessuno
- **Output:** oggetto JSON nel formato indicato

// OUTPUT - JSON

```
{
  "domains": [
    "esempio_1.it",
    "esempio_2.it",
    ...
  ]
}
```

Obiettivo 6: API REST

GET /gold_standard

- Restituisce l'entry del GS per l'URL dato in input. Caricando le informazioni dal DB
- **Input:** "url": str
- **Output:** oggetto JSON nel formato indicato.
- **Errori:**
 - Se il dominio non è supportato
 - L'URL non è nel GS (non è nel DB)

// OUTPUT - JSON

```
{  
  "url": "https://...",  
  "domain": "esempio.it",  
  "title": "Titolo estratto dalla pagina web",  
  "html_text": "Html estratto dalla  
                pagina web, input del parser",  
  "gold_text": "Testo pulito, estratto  
                manualmente dalla pagina web,  
                no MARKDOWN"  
}
```

Obiettivo 6: API REST

GET /gold_standard_urls

- Restituisce la lista degli url di un GS per un dominio specifico. Caricando le info dal DB
- **Input: "domain": str**
(lo stesso usato nella lista get domains)
- **Output:** oggetto JSON nel formato indicato.
- **Errori:**
 - Se il dominio non è supportato.

// OUTPUT - JSON

```
{
  "gold_standard_urls":
  [
    "https://esempio_1.com/pagina_1",
    "https://esempio_1.com/pagina_1",
    ...
    "https://esempio_1.com/pagina_n"
  ]
}
```

Obiettivo 6: API REST

POST /evaluate

- Dato il risultato del parsing di un documento e il corrispettivo testo del GS restituisce le metriche di evaluation
Rimuovere il markdown in fase di evaluation, confrontare solo il contenuto.
- **Input:** {"parsed_text": str, "gold_text": str}
- **Output:** oggetto JSON nel formato indicato.
token_level_eval con **precision**, **recall** e **f1** è obbligatoria

// OUTPUT - JSON

```
{
  "token_level_eval": {
    "precision": 0.87,
    "recall": 0.91,
    "f1": 0.89,
  },
  "x_eval": {
    ...
  }
}
```

Obiettivo 6: API REST

POST /evaluate_judge

- Dato il risultato del parsing di un documento e il corrispettivo testo del GS restituisce la valutazione del judge.
Rimuovere il markdown in fase di evaluation, confrontare solo il contenuto.
- **Input:** {"parsed_text": str, "gold_text": str}
- **Output:** oggetto JSON nel formato indicato.
- Potete aggiungere **altri campi** per evaluation più strutturate ma *"model_name"*, *"judge_score"* e *"judge_feedback"* sono **obbligatori**

// OUTPUT - JSON

```
{  
  "model_name": "minerva:7b",  
  "judge_score": 1, # intero tra 1 e 5  
  "judge_feedback": "Feedback testuale  
                    restituito dal modello",  
  "altre info" : ...  
}
```

Obiettivo 6: API REST

GET /full_gs_eval

- Restituisce la **valutazione complessiva** del **Gold Standard** per il dominio dato in input. Esegue il **parsing** di tutte le sue entry e aggrega i risultati tramite **media** delle **metriche** e del **judge_score**, calcolate sui singoli elementi del GS. Rimuovere il markdown in fase di evaluation, confrontare solo il contenuto.
Usare **HTML statico** salvato nel **DB**, ma calcolare il resto.
- **Input: "domain": str**
(lo stesso usato nella lista get domains)
- **Output:** oggetto JSON nel formato indicato.
- **Errori:**
 - Se il dominio non è supportato.

```
// OUTPUT - JSON
{
  "token_level_eval": {
    "precision": 0.87,
    "recall": 0.91,
    "f1": 0.89,
  },
  "judge_score": 2.6,
  "x_eval": {
    ...
  }
}
```

Obiettivo 6: API REST

POST /add_web_resource & /add_gold_standard

- **POST /add_web_resource**
 - Aggiunge una risorsa web nella tabella **web_resources** con l'HTML fornito in input
 - **Input:** {"url": str, "html_text": str}
 - **Output:** oggetto JSON nel formato indicato
- **POST /add_gold_standard**
 - Aggiunge un'entry nella tabella **gold_standard**, la **web_resource** corrispondente deve già esistere nel DB
 - **Input:** {"url": str, "gold_text": str}
 - **Output:** oggetto JSON nel formato indicato
 - **Errori:** URL non presente in **web_resources**

```
// OUTPUT - JSON
```

```
{"status": "ok"} # oppure "error"
```

```
// OUTPUT - JSON
```

```
{"status": "ok"} # oppure "error"
```

Obiettivo 6: API REST

DELETE /web_resource & /gold_standard

- **DELETE /web_resource**
 - Rimuove una risorsa web dalla tabella web_resources e a cascata tramite FK, il gold_standard associato
 - **Input:** {"url": str}
 - **Output:** oggetto JSON nel formato indicato
- **DELETE /gold_standard**
 - Rimuove solo l'entry dalla tabella gold_standard, lasciando intatta la web_resource associata
 - **Input:** {"url": str}
 - **Output:** oggetto JSON nel formato indicato
- **Errori:** URL non presente nel GS

// OUTPUT - JSON

```
{"status": "ok"} # oppure "error"
```

// OUTPUT - JSON

```
{"status": "ok"} # oppure "error"
```

Obiettivo 6: API REST

GET /db_stats

- Restituisce **stats** e **valutazione** complessiva dei **dati** che sono **salvati** nel **DB** per ogni dominio. Le metriche obbligatorie e il judge score devono essere riportati sempre in questa statistica. Questa funzione utilizza tutti **dati pre-calcolati e salvati nel DB**.
- **Input:** nessuno
- **Output:** oggetto JSON nel formato indicato.

NOTA: ricordate di popolare il DB in fase di inizializzazione dei vari docker

```
// OUTPUT - JSON
{
  "web_resources": {
    "esempio_1.it": 15,
    "esempio_2.it": 25
  },
  "gold_standard": {
    "esempio_1.it": 15,
    "esempio_2.it": 25
  },
  "avg_eval": {
    "esempio_1.it": {
      "token_level_eval": { "f1": 0.82, ...
      "metric_2": { "score": 0.91}
    },
    "esempio_2.it": {
      "token_level_eval": { "f1": 0.99, ...
      "metric_2": { "score": 0.95}
    }
  },
  "avg_eval_judge": {
    "esempio_1.it": {
      "judge_score": 2.4,
      "metric_2": 0.50
    },
    "esempio_2.it": {
      "judge_score": 4.7,
      "metric_2": 0.80
    }
  }
}
```

Obiettivo 6: API REST

GET /db_schema

- Restituisce lo schema del DB. Le tabelle con le relative colonne, specificando tipo dei dati, chiavi primarie (PK), chiavi esterne (FK)
- **Input:** nessuno
- **Output:** oggetto JSON nel formato indicato

// OUTPUT - JSON

```
{
  "web_resources": {
    "url": "varchar(2048), PK",
    "domain": "varchar(255)",
    "title": "varchar(2048)",
    "html_text": "longtext",
    "created_at": "datetime"
  },
  "gold_standard": {
    "url": "varchar(2048), PK, FK(web_resources.url)",
    "gold_text": "longtext",
    "created_at": "datetime"
  },
  ...
}
```

Obiettivo 6: API REST

GET /status

- Restituisce lo **stato dei componenti del sistema**.
I valori possibili per ogni componente sono **"ok"** se il componente è raggiungibile e funzionante, **"error"** altrimenti
- **Input:** nessuno
- **Output:** oggetto JSON nel formato indicato.
L'endpoint restituisce sempre HTTP 200, è il contenuto del JSON a indicare lo stato reale dei componenti, non il codice HTTP

// OUTPUT - JSON

```
{  
  "backend": "ok",  
  "database": "error",  
  "ollama": "ok"  
}
```

Obiettivo 7: Web UI

Interfaccia Web

- **L'interfaccia web** permette di **interagire** con l'intera **pipeline** senza utilizzare direttamente le API REST
- Composta da una **home page** e **tre sezioni principali**, ognuna con una responsabilità specifica:
 - **testare** il parser su un URL
 - **costruire** e gestire il **Gold Standard**
 - **monitorare** le **statistiche** aggregate del sistema

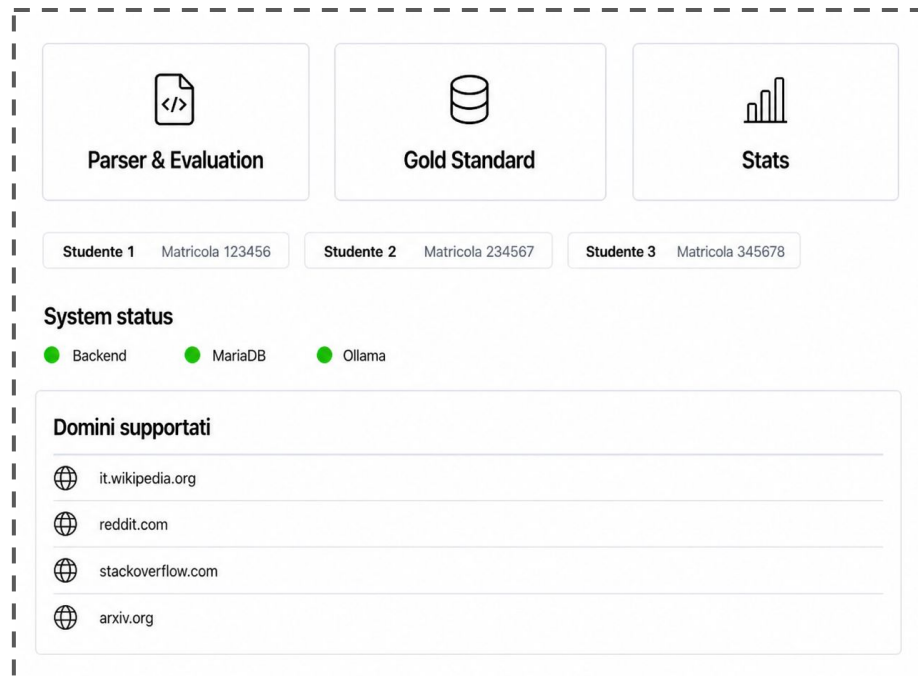
Obiettivo 7: Web UI

Interfaccia Web – Home Page

Pagina principale dell'interfaccia web:

- **Navigazione** verso le **3 pagine**
- **Matricole** studenti del gruppo
- **Stato** del sistema (backend, db e ollama)
- **Domini** supportati dal sistema

NOTA: le immagini della UI mostrate sono puramente esemplificative, avete totale libertà nella scelta di stile e layout del frontend



Obiettivo 7: Web UI

Interfaccia Web – Pagina Parser & Evaluation

Pagina di valutazione qualitativa dei parser:

- Permette di **testare un parser** inserendo un **URL** o selezionare quelli nel GS dal menu a tendina
- Modalità **Live/Local**: scarica la pagina in tempo reale dal web oppure usa quella nel DB
- Mostra il **confronto tra html_text grezzo e parsed text** in Markdown
- Se l'URL è presente nel **Gold Standard**
 - **Confronta** anche il risultato con il **gold_text**
 - **Visualizza** tutte le **metriche** di valutazione implementate
 - Include anche una **valutazione** tramite LLM **Judge**, con score e feedback testuale

URL di test

Modalità

Live Local

Scarica la pagina in tempo reale dal web Usa HTML già salvato nel database

Esegui parsing e valutazione

Input HTML

```
<!DOCTYPE html>
<html lang="it" class="client-nojs">
<head> ... </head>
<body class="mediawiki ltr">
  <div id="content" class="mw-body" role="main">
    ...
  </div>
</body>
</html>
```

Output parsed

```
# Intelligenza artificiale

Intelligenza artificiale (IA) è un insieme di tecnologie che consentono alle macchine di simulare capacità cognitive umane.

## Storia
- Anni '50: primi studi e programmi simbolici
- Anni '80: sistemi esperti e reti neurali
- Anni 2000+: apprendimento automatico e big data

## Applicazioni
- Elaborazione del linguaggio naturale
- Visione artificiale
- Robotica e automazione

...
```

Gold text

Intelligenza artificiale (IA) è un insieme di tecnologie che consentono alle macchine di simulare capacità cognitive umane.

...

Applicazioni

- Elaborazione del linguaggio naturale
- Visione artificiale
- Robotica e automazione

...

Precision 0.89 Recall 0.93 F1 0.91 LLM Judge 4/5

Giudizio complessivo

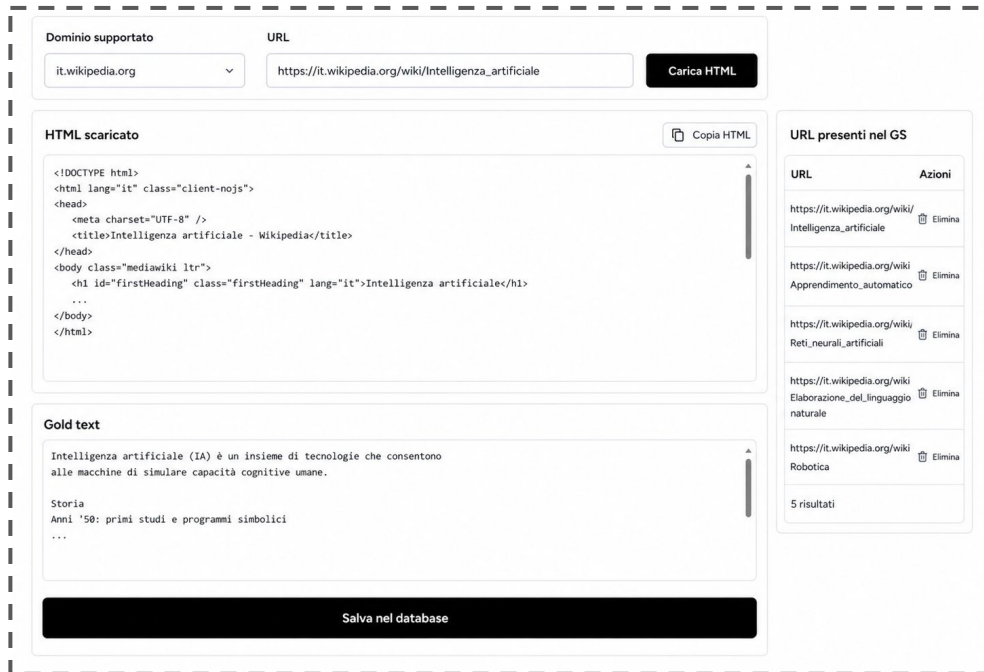
Buona qualità generale: il parser mantiene struttura e contenuti principali, con lievi differenze di formattazione.

Obiettivo 7: Web UI

Interfaccia Web – Pagina Gold Standard Builder

Pagina che permette di **gestire il Gold Standard** nel DB, aggiungendo e rimuovendo dati:

- **Selezione del dominio** tra quelli supportati
- Inserimento dell'URL: il sistema **scarica l'HTML** e lo mostra nella pagina. È possibile copiare l'HTML statico mostrato per aprirlo in un browser per copiare manualmente il testo informativo per costruire i GS
- Area di testo per **inserire il gold_text** copiato dalla pagina
- Al submit, **HTML** e **gold_text** vengono salvati insieme nel **database**
- Inoltre, la pagina mostra la **lista degli URL** già presenti nel GS per il dominio selezionato, con **possibilità di eliminare** ogni entry



The screenshot displays the Gold Standard Builder web interface, which is used for managing Gold Standard data in a database. The interface is divided into several sections:

- Domain Support:** A dropdown menu showing "it.wikipedia.org".
- URL:** A text input field containing "https://it.wikipedia.org/wiki/Intelligenza_artificiale".
- Carica HTML:** A button to trigger the download of HTML content from the specified URL.
- HTML scaricato:** A text area displaying the downloaded HTML content, including the document type, language, charset, and title. A "Copia HTML" button is located to the right.
- Gold text:** A text area for entering the Gold Standard text, which is extracted from the HTML content. It shows the first paragraph of the article on Artificial Intelligence.
- Salva nel database:** A button to save the entered Gold Standard text to the database.
- URL presenti nel GS:** A table listing URLs already present in the Gold Standard for the selected domain. Each entry includes the URL and an "Elimina" (Delete) button.

URL	Azioni
https://it.wikipedia.org/wiki/Intelligenza_artificiale	Elimina
https://it.wikipedia.org/wiki/Apprendimento_automatico	Elimina
https://it.wikipedia.org/wiki/Reti_neurali_artificiali	Elimina
https://it.wikipedia.org/wiki/Elaborazione_del_linguaggio_naturale	Elimina
https://it.wikipedia.org/wiki/Robotica	Elimina
5 risultati	

Obiettivo 7: Web UI

Interfaccia Web – Pagina Stats

Pagina che mostra una **panoramica** aggregata di **tutti i domini** in un'unica vista:

- **Risorse nel DB:** numero di **entry** in `web_resources` e `gold_standard` per ciascun dominio
- **Metriche medie** calcolate sul Gold Standard di ciascun dominio
- **Judge score medio** valutazione qualitativa aggregata per ciascun dominio
- Siete liberi di aggiungere **ulteriori statistiche** che ritenete utili

Obiettivo 7: Web UI

Requisiti Generali

- Tecnologie obbligatorie: **Jinja2** per il templating e **HTML/CSS** per il frontend. È possibile utilizzare tecnologie aggiuntive purché non appesantiscano inutilmente il sistema
- La UI gira in un **container Docker** dedicato coordinato tramite **Docker Compose**
- Tutta la **logica** passa **esclusivamente** tramite le **API REST** del **backend**. Nella specifica del backend ci sono tutte le funzioni necessarie.
- La UI deve **gestire** e mostrare chiaramente tutti gli **errori** restituiti dalle **API** (es. dominio non supportato, URL non raggiungibile, URL non presente nel DB)

Obiettivo 8: Containerizzazione

Architettura multi-container con Docker Compose

Il sistema è composto da 4 container Docker orchestrati tramite **Docker Compose**, avviabili tramite *docker-compose.yaml* con un unico comando: *"docker compose up --build"*

- **Backend:** server FastAPI con i parser, le API REST e la logica di evaluation.
Esposto sulla porta 8003
- **Frontend:** server Jinja2/HTML per la Web UI. Esposto sulla porta 8004
- **Database:** MariaDB con i dati persistenti caricati all'avvio del sistema.
Esposto sulla porta 3306
- **Ollama:** server LLM locale con il modello importato automaticamente all'avvio.
Esposto sulla porta 11434

NOTA: Le **porte** indicate sono **obbligatorie** e devono essere **esposte** esattamente come specificato. I test automatici le useranno per comunicare con il sistema. Usare solo **path relativi**, nessun path assoluto.

Consegna



Consegna

Struttura delle Directory del Progetto

- **docker-compose.yaml**: avvia l'intero sistema con un solo comando (tutti e 4 i docker)
- **domains.json**: lista dei domini assegnati
- **backend/**: server FastAPI, codice necessario per i parser e la valutazione del parsing
- **frontend/**: interfaccia web HTML/Jinja2 con template e codice necessario per la UI
- **gs_data/**: Gold Standard JSON per ciascun dominio
- **mariadb_data/**: eventuali file necessari per il database di MariaDB
- **ollama_data/**: eventuali file necessari per Ollama
- **report.pdf**: max 2/3 pagine, redatto in LaTeX

```
progetto/
├── docker-compose.yaml
├── domains.json
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src/
│       └── ... (file .py)
├── frontend/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src/
│       └── ... (file .py)
│       └── templates/
├── gs_data/
│   ├── dominio_x_gs.json
│   └── dominio_y_gs.json
├── ollama_data/
├── mariadb_data/
└── report.pdf
```

Consegna

Singolo Zip

- Consegnare un **singolo file zip** che è lo zip della **cartella** del vostro **progetto**, con il nome che include tutte le matricole dei membri del gruppo:
matricola1_matricola2_matricola3_lab_progetto.zip
- La **struttura** delle **directory** all'interno del file zip deve essere in linea con quella **specificata** nelle **slide precedenti**
- Potete usare più file, cartelle e sottocartelle se li repute utili a organizzare meglio il progetto

Test automatico

Test del Backend

- Vi forniremo nei prossimi giorni su Google Classroom uno **script** che avvierà un **test automatico** di tutti gli **endpoint** definiti nella specifica del **WEB Server** del progetto
- Dovete **esporre il backend** del vostro **progetto** sulla porta **8003**. Lo script di testing, eseguito fuori dai Docker container del vostro progetto, deve poter accedere a quella porta per eseguire il test automatico (non modificare la porta nello script di test)
- Dovete **esporre il frontend** del vostro **progetto** sulla porta **8004** (anche se non viene esplicitamente testato)
- Dovete **esporre anche MariaDB 3306 e Ollama 11434**
- Il test deve funzionare subito dopo il comando: ***docker compose up --build***
Non ci devono essere passaggi intermedi tra questa inizializzazione automatica e l'esecuzione dello script di test



Test automatico

Test del Backend

- Dovete riuscire ad **eseguire** l'intero **script** di **test** senza modifiche arrivando a vedere il messaggio che indica che tutti i test sono stati superati:
"ALL TESTS PASS!"
Indispensabile prima di effettuare la consegna
- **Non** dovete **modificare** lo **script** di **test**, dato che noi lo useremo esattamente così com'è per valutare i progetti
- Se pensate di aver **trovato** un **errore** nello script di test **comunicarlo** tramite **Google Classroom**



Test automatico

Test del Frontend

- Ricordate di **testare** voi stessi anche il **frontend** del vostro **progetto**, dopo aver eseguito i test automatici con successo e prima di consegnare
- Il frontend **non viene valutato automaticamente** dallo script di test
- **Non** dovete **consegnare** lo **script di test** insieme al vostro progetto (è uguale per tutti)



Test automatico

Docker Compose

- Il vostro **progetto** verrà **eseguito** tramite il comando **“docker compose up --build”** eseguito da terminale nella directory del file **docker-compose.yaml** che deve essere nella root del progetto, come indicato nelle slide precedenti
- Il file **docker-compose.yaml** contiene le istruzioni necessarie per avviare l'intero progetto con il comando **“docker compose up --build”**
 - I due moduli **frontend** e **backend** separati in cartelle diverse, ognuna con i suoi file **requirements.txt** e **Dockerfile**, necessari per la costruzione dei rispettivi Docker container
 - I due servizi **MariaDB** e **Ollama**



Consegna

Step Fondamentali – Docker Compose

- Il comando **docker compose up --build** deve funzionare su un qualsiasi computer, quindi non si possono usare path assoluti ma solo path relativi.
Prima di consegnare spostate il vostro progetto in un'altra cartella nel vostro computer e controllare che **docker compose up --build** funzioni ancora ***senza cambiare nulla*** per assicurarvi che funzioni anche sul nostro computer che useremo per le correzioni.
- Potete sviluppare su Mac/Windows ma noi eseguiremo il comando **docker compose up --build** su Linux (Ubuntu), quindi se state sviluppando su Mac/Windows dovete controllare che questo comando funzioni correttamente anche su **Ubuntu** prima di consegnare (usate una **macchina virtuale**, vedere slide successive)
- Ricordate di eseguire anche il test automatico e di testare il frontend dopo ognuno di questi test



Consegna

Step Fondamentali – Macchina Virtuale

- Scaricate l'ISO di **Ubuntu 22.04.5** (desktop) da <https://releases.ubuntu.com/jammy/>
- Installatelo o come **macchina virtuale** o come **dual boot**. Fate il test finale in un nuovo dual boot o in nuova macchina virtuale, non nel PC in cui avete sviluppato il progetto, questo vi garantisce che funzionerà anche nel nostro ambiente.
- Tutte le **istruzioni** seguenti vanno **eseguite** dentro la **macchina virtuale** o il **dual boot**.
- Installate **docker** e **docker compose** con le versioni corrette (vedi Google Classroom e form di consegna).
- Navigate da terminale nella **cartella** del vostro **esonero** esattamente come la consegnerete, salvatene anche una copia dato che la dovete consegnare come è prima di **eseguire i test** e non dopo



Consegna

Step Fondamentali – Permessi, Test e Consegna Zip

- Eseguire il comando “**sudo chmod 777 -R .**” per dare i permessi necessari e potenzialmente altre cartelle necessarie.
- Eseguire il comando “**sudo docker compose up --build**” e aspettare che l’inizializzazione termini
- Eseguire lo script di test che deve funzionare dando in output “**ALL TESTS PASS**”
- Consegnare la copia della cartella così come era prima di eseguire il test compressa in un unico file **.zip**
- **Il progetto va consegnato su Google Classroom tramite l’apposito form, che sarà pubblicato nelle settimane precedenti alla scadenza**



Report del Progetto

Informazioni Generali

- Realizzare un **report** in un file **report.pdf** (anche questo incluso nell'archivio zip) di massimo **2 pagine** (fontsize 11) che descrive il vostro progetto:
 - **Obiettivo del Progetto:** Brevissimo riassunto della funzionalità del progetto (1-2 frasi)
 - **Organizzazione del Codice:** Come è strutturato il vostro codice e una spiegazione delle responsabilità dei vari moduli (no snippet di codice)
 - **Valutazione Parser:** Analisi dei dati, tecniche di valutazione dei parser, risultati ottenuti ed eventuali problematiche riscontrate
 - **Valutazione dei Judge:** Selezione del modello e del prompt ideale
 - **Organizzazione del DB:** Come avete strutturato il vostro database
- **Bonus/Extra:** Qualora desideriate inserire contenuti ulteriori rispetto ai requisiti del progetto, è possibile aggiungere una **terza pagina** finale in una sezione intitolata "**Bonus/Extra**" (ad esempio per sottodomini/domini aggiuntivi o altre aggiunte rilevanti).
- Il report deve essere scritto in **LaTeX** partendo da questo template online: [link](#)
 - Non includere l'abstract, partite direttamente con la prima **`\begin{section}`**
 - Decomentare **`\aclfinalcopy`** eliminando il % davanti a quella riga del file **.tex** per poter mostrare il vostro nome e cognome (altrimenti verrà mostrato "anonymous submission")



Valutazione

Criteri di valutazione

- Saranno valutati:
 - L'effettivo funzionamento del progetto (bloccante: non si passa se il progetto non funziona e non passa il test automatico)
 - La qualità dei parser e dei GS
 - La corretta progettazione del DB e del prompt per il Judge
 - L'organizzazione del progetto (cartelle, file, ecc.)
 - La qualità e la pulizia del codice (inclusi i commenti e i type hint)
 - La modularità del progetto: la corretta suddivisione in moduli e l'uso della OOP
 - La corretta gestione dei casi limiti o di errore
 - La qualità della UI
 - Il report
- Il plagio (sia chi fa copiare che chi copia) porterà all'annullamento del progetto



Progetto Personalizzato Tesi

Riservato ai progetti migliori



Progetti personalizzati

Indicazioni generali per i progetti migliori

- Gli studenti con i migliori homework 1 (vedi [ranking](#)) potranno decidere di optare per uno dei progetti personalizzati
- La scelta è possibile solo entro la fine del corso
- E' possibile consegnare il progetto personalizzato a ogni appello come il progetto base
- Il progetto deve comunque essere prima discusso direttamente con il prof. Navigli
- Il progetto potrà essere presentato direttamente come tesi di laurea triennale
- E' possibile creare un sottogruppo del gruppo iniziale o discutere la formazione di nuovi gruppi, sempre a partire dai migliori studenti

Progetto Personalizzato Tesi (1b)

Introduzione

- **LLM2Graph**: trasformare l'output di un LLM in un grafo navigabile

BabelNet

Voce Discussione

Leggi Modifica Modifica wikitesto Cronologia Strumenti

BabelNet è una **rete semantica** multilingue, un **grafo di conoscenza** e un'**ontologia** lessicalizzata.^{[1][2]} BabelNet è stata creata integrando automaticamente la più grande **enciclopedia** multilingue – ovvero **Wikipedia** – con il più noto **lessico** della **lingua inglese** – ovvero **WordNet**. L'integrazione è stata effettuata per mezzo di una mappatura automatica. Le voci mancanti nelle altre lingue sono state ottenute con l'ausilio di tecniche di **traduzione automatica**. Il risultato è un "dizionario enciclopedico" che fornisce **concetti** e voci enciclopediche **lessicalizzate** in molte lingue, collegati tra loro da grandi quantità di relazioni semantiche. Analogamente a WordNet, BabelNet raggruppa le **parole** in lingue diverse in insiemi di **sinonimi**, chiamati **Babel synset**. Per ciascun Babel synset, BabelNet fornisce definizioni testuali (chiamate **glosse**) in diverse lingue, ottenute da WordNet e da Wikipedia.

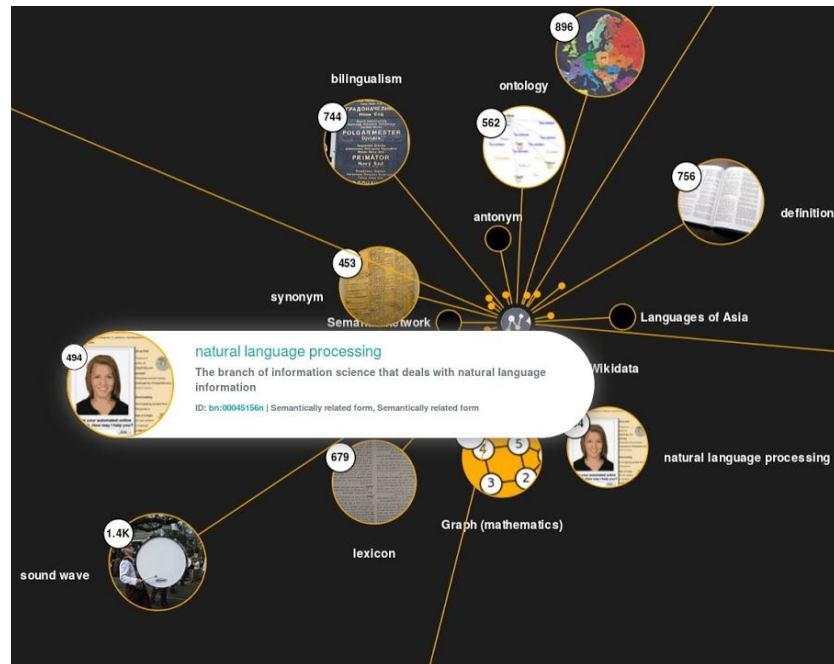
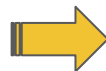
BabelNet è stato citato sul **Time magazine**^[3] come il rappresentante di una nuova era di risorse lessicografiche computazionali del ventunesimo secolo.

Statistiche

[modifica | modifica wikitesto]

La versione 5.3 di BabelNet, rilasciata nel dicembre 2023, copre 600 **lingue**. Contiene quasi 23 milioni di **synset** e circa 1,7 miliardi di **sensi** (indipendentemente dalla loro lingua). Ciascun Babel synset contiene in media 2 sinonimi per lingua, cioè sensi linguistici. Il knowledge graph include tutte le relazioni lessico-semantiche di WordNet (**iperonimia** e **iponimia**, **meronimia** e **olonomia**, **antonomia** e **sinonimia**, ecc., per un totale di circa 364.000 archi relazionali), così come una relazione generica di correlazione proveniente da Wikipedia (per un totale di circa 1,9 miliardi di archi).^[1] La versione 5.3 associa inoltre circa 61 milioni di immagini ai Babel synset. Fornisce una copia di ogni pagina di Wikipedia multilingue.

BabelNet sito web	
	
BabelNet	
URL	www.babelnet.org
Tipo di sito	Dizionario multilingue e rete semantica online
Lingua	Multilingue
Registrazione	facoltativa
Commerciale	No
Creato da	Roberto Navigli
Stato attuale	In progresso
Slogan	Search, translate, ...



SAPIENZA
UNIVERSITÀ DI ROMA

Progetto Personalizzato Tesi (1b)

Introduzione

- **LLM2Graph:** trasformare l'output di un LLM in un grafo navigabile
 - Estrarre entità dal testo
 - con LLM, spaCy o modello NER
 - Estrarre relazioni tra entità, ovvero triple soggetto - relazione - oggetto
 - con LLM o sistemi di relation extraction
 - Normalizzare e disambiguare le entità
 - Collegare entità duplicate o ambigue a identificativi unici (Wikidata QID o BabelNet synset)
 - Convertire le triple in un grafo
 - Salvare nodi e archi in Neo4j o NetworkX
 - Validare il grafo
 - Controllare inconsistenze, relazioni inventate, nodi duplicati e triple non supportate dal testo
 - Costruire una visualizzazione navigabile
 - Usare strumenti come Neo4j Bloom, Cytoscape.js, D3.js, GraphDB o Streamlit + PyVis
 - Aggiungere interazione e ricerca
 - Filtri per tipo di nodo, ricerca testuale, espansione dei vicini e visualizzazione delle fonti
 - Valutare il risultato

Progetto Personalizzato Tesi (2b)

Agenti per Minerva

- **Sviluppare agenti da integrare nel framework agentico di Minerva:**
 - Calcolatrice
 - Matematica simbolica (es. con SymPy)
 - Analisi dati in Python (con pandas)
 - Visualizzazione dati (grafici)
 - Esecuzione codice Python
 - Query SQL (linguaggio naturale → database)
 - Pulizia dati
 - Ricerca web
 - Riassunto notizie
 - Interrogazione knowledge graph
 - Multi-API (meteo, finanza, ecc.)
 - Pianificazione
 - Monitoraggio finanza (azioni/crypto)
 - Generazione doc
 - Generazione fogli excel



Progetto Personalizzato Tesi (3b)

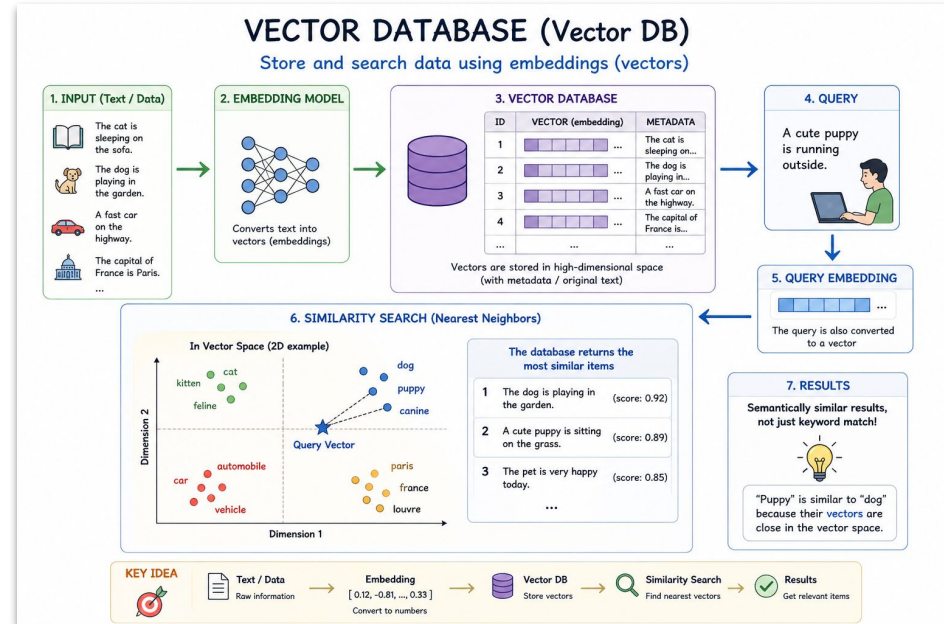
Introduzione

- Uno studio sperimentale dei **vector database**

- FAISS
- Chroma
- Weaviate
- Milvus
- Qdrant
- Vespa
- Mongo DB Atlas Vector Search

- Valutazione di prestazioni

- In termini di velocità
- In termini di spazio in RAM / disco
- Per grandi / grandissimi volumi

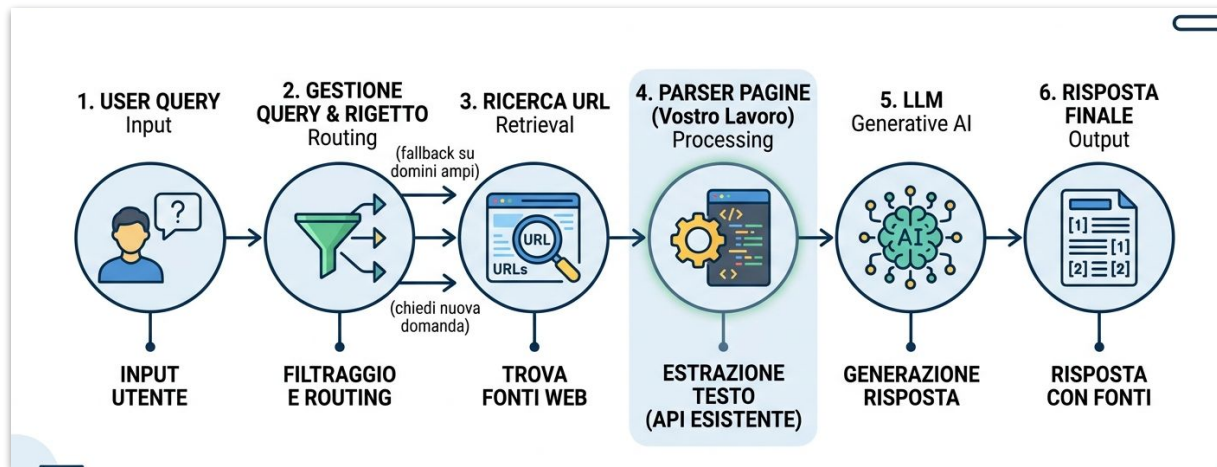


Progetto Personalizzato Tesi (4b)

Chat Bot con info dal web

● Chat Bot - Di Dominio

- Consentire all'utente di porre al proprio sistema una query
- Assicurarsi che la query dell'utente possa essere risolta sfruttando i domini disponibili
- Ricercare online pagine (nei domini disponibili) che possano contenere informazioni utili
- Parsare le pagine (sfruttando i parser dell'esonero) e fornire ad un LLM query+pagine
- Restituire risposte all'utente



Progetto Personalizzato Tesi (4b)

Chat Bot con info dal web – Sfide

- **Chat Bot – Di Dominio**

- **Comprensione e disambiguazione della domanda**

Gestire richieste ambigue e chiedere eventualmente chiarimenti all'utente

- **Filtro delle domande fuori dominio**

Definire criteri di rigetto e valutare il comportamento in caso di domanda non accettabile

- **Recupero e selezione delle fonti**

Individuare pagine web pertinenti e classificarle in base alla loro utilità (valutare anche altre domini)

- **Gestione del contesto e del prompt**

Riassumere o selezionare contenuti rilevanti per non saturare la context window delle modello

- **Qualità e affidabilità della risposta**

Citare le fonti, stimare l'affidabilità e gestire risposte parziali o incerte

Per le tesi personalizzate

Incontro previsto per il 3 giugno

Per chi vuole consegnare il progetto da luglio in avanti (non essendoci i tempi tecnici per un incontro relativo a chi consegnerà a giugno)

